TP2

Analyse de la complexité des algorithmes

Nom et prénom : KAZALO Wilfride
Licence GL - Semestre 6 - Année académique 2025 - 2026
Cours : Conception et analyse des algorithmes - M. KARABOU

Conventions d'analyse

On note n la taille de l'entrée. On utilise le modèle usuel où une comparaison, une affectation, une opération arithmétique sur un indice et l'affichage d'une valeur comptent comme des opérations élémentaires de coût constant. La complexité en espace désigne la mémoire auxiliaire, sans compter les données d'entrée ni le stockage de la sortie. Les affichages sont produits au fur et à mesure. La notation Θ donne un ordre de grandeur exact à un facteur constant près ; elle implique aussi la borne supérieure O correspondante.

1 Algorithme avec une boucle simple

Question 1.1 - Complexité en temps

La boucle parcourt les entiers de 1 à n inclus et réalise donc exactement n itérations. Chaque itération exécute un affichage de coût constant dans le modèle retenu. Le temps total est $T(n) = an + b$, où a et b sont des constantes. La complexité est donc $\Theta(n)$, et en particulier $O(n)$. Elle est identique dans tous les cas puisque le nombre d'itérations dépend uniquement de n.

Question 1.2 - Complexité en espace

L'algorithme conserve seulement l'indice i et un nombre constant de variables de contrôle. En Python 3, range ne construit pas une liste contenant tous les entiers. La mémoire auxiliaire est donc $\Theta(1)$, soit $O(1)$. Les valeurs affichées ne sont pas accumulées en mémoire par cet algorithme.

2 Algorithme avec deux boucles imbriquées

Question 2.1 - Complexité en temps

La boucle extérieure effectue n itérations. Pour chacune, la boucle intérieure effectue également n itérations. L'instruction print(i, j) est donc exécutée $n \times n = n^2$ fois. Le temps total est $\Theta(n^2)$, soit $O(n^2)$, dans tous les cas.

Question 2.2 - Complexité en espace

Seuls les indices i et j et les variables de contrôle des boucles sont conservés. Aucun tableau de n^2 couples n'est créé. La mémoire auxiliaire est donc $\Theta(1)$, soit $O(1)$, malgré les n^2 affichages.

3 Recherche dichotomique

Question 3.1 - Complexité en temps

Le tableau doit être trié. À chaque itération, on compare la valeur recherchée à l'élément du milieu, puis on élimine au moins la moitié de la zone restante. Après k itérations, la zone contient au plus environ $n / 2^k$ éléments. Il suffit donc d'un nombre d'itérations proportionnel à $\log_2(n)$ pour terminer.

Dans le pire cas, la complexité est $\Theta(\log n)$, soit $O(\log n)$. Le meilleur cas est $\Theta(1)$, lorsque x est trouvé dès la première comparaison au milieu du tableau. Pour une recherche réussie où chaque position du tableau est équiprobable, le temps moyen est $\Theta(\log n)$. Le coût d'un éventuel tri préalable n'est pas compris dans cette recherche.

Question 3.2 - Complexité en espace

Cette version est itérative. Elle utilise un nombre constant de variables : low, high et mid. Elle ne copie aucune portion du tableau et n'utilise pas de pile d'appels récursifs. Sa mémoire auxiliaire est donc $\Theta(1)$, soit $O(1)$.

4 Tri par insertion

Question 4.1 - Complexité en temps dans le pire cas

Le pire cas apparaît lorsque le tableau contient des valeurs distinctes triées dans l'ordre décroissant. Pour insérer l'élément d'indice i, il faut alors décaler les i éléments qui le précèdent. Le nombre total de décalages vaut $1 + 2 + \ldots + (n - 1) = n(n - 1)/2$. Les comparaisons et affectations ont le même ordre de grandeur. La complexité dans le pire cas est donc $\Theta(n^2)$, soit $O(n^2)$.

À titre de comparaison, le meilleur cas est $\Theta(n)$ lorsque le tableau est déjà trié : aucun décalage n'est nécessaire. Pour une permutation aléatoire de valeurs distinctes, le cas moyen est $\Theta(n^2)$.

Question 4.2 - Complexité en espace

Le tri modifie directement le tableau d'entrée. Il utilise seulement les variables i, j et key, sans tableau supplémentaire. Sa mémoire auxiliaire est $\Theta(1)$, soit $O(1)$: il s'agit d'un tri en place.

5 Voyageur de commerce par force brute

Question 5.1 - Complexité en temps

Le programme parcourt les n! permutations des n villes. On suppose que calculate_cost calcule le coût d'un circuit en additionnant ses n déplacements, y compris le retour à la ville de départ, et que le coût de chaque déplacement est accessible en temps constant. Le calcul du coût d'une permutation prend alors $\Theta(n)$. Le temps total est donc $\Theta(n \times n!)$, soit $O(n \times n!)$.

La fonction calculate_cost n'étant pas définie dans l'énoncé, cette hypothèse doit être précisée. Le terme $O(n!)$ décrit le nombre de circuits examinés, mais omet le travail linéaire nécessaire pour évaluer chaque circuit dans l'hypothèse retenue. Le programme ne s'arrête pas après avoir trouvé un bon circuit : il examine toujours toutes les permutations. Son ordre de grandeur reste donc le même dans les meilleurs et pires cas.

Question 5.2 - Complexité en espace

En supposant que permutations est un générateur, par exemple itertools.permutations en Python, les permutations sont produites une par une. Le générateur et la permutation courante occupent $\Theta(n)$ mémoire. Le meilleur chemin mémorisé contient également n villes ; conserver ces structures simultanément reste linéaire. Si calculate_cost utilise une mémoire constante ou linéaire, la mémoire auxiliaire totale est $\Theta(n)$, soit $O(n)$.

Si toutes les permutations étaient d'abord stockées dans une liste, il faudrait au contraire $\Theta(n \times n!)$ mémoire. Ce stockage n'apparaît pas dans l'algorithme donné. Une éventuelle matrice de distances déjà fournie en entrée n'est pas comptée dans la mémoire auxiliaire.

6 Choix du bon algorithme

Question 6.1 - Choix pour $n = 10^6$

Le choix dépend d'abord de l'opération demandée : le tri par insertion et le tri fusion ordonnent des données, tandis que la recherche dichotomique cherche une valeur dans des données déjà triées. Ces trois algorithmes ne résolvent donc pas exactement le même problème.

Algorithme	Temps pour de grandes entrées	Utilisation
Tri par insertion	$\Theta(n^2)$ en moyenne et au pire ; $\Theta(n)$ au mieux	Adapté surtout aux petits tableaux ou aux données presque triées.
Tri fusion	$\Theta(n \log n)$ pour la version classique	Choix général pour trier un grand tableau ; $\Theta(n)$ mémoire auxiliaire sur tableau.
Recherche dichotomique	$\Theta(\log n)$ au pire ; $\Theta(1)$ au mieux	Recherche rapide d'une valeur dans un tableau déjà trié ; $\Theta(1)$ mémoire auxiliaire pour la version itérative.

Pour trier un million d'éléments sans hypothèse favorable sur leur ordre initial, je choisirais le tri fusion. Comme $\log_2(10^6) \approx 19{,}93$, $n \log_2(n)$ vaut environ 2×10^7, contre environ 5×10^{11} décalages pour le tri par insertion dans son pire cas. Ce sont des ordres de grandeur du travail, pas des durées d'exécution.

Pour rechercher une valeur dans un tableau d'un million d'éléments déjà triés, je choisirais la recherche dichotomique : elle demande au plus environ 20 itérations. Si les données ne sont pas triées et qu'un grand nombre de recherches est prévu, on peut les trier une seule fois par tri fusion, puis utiliser la recherche dichotomique. Pour k recherches, le coût total vaut $O(n \log n + k \log n)$.